The Journal of Open Source Software

BeeMesh: A lightweight volunteer distributed computing framework for scientific workloads

Dhanush Vittal Shenoy[1], **Dheeraj Vittal Shenoy**[2], **and Steven H Frankel**[3]

1 Post-doctoral researcher, Technion, Haifa, Israel 3200003 **2** Doctoral researcher, IFCA, CSIC/UC, Spain **3** Faculty of Mechanical engineering, Technion, Haifa, Israel 3200003

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

BeeMesh is a lightweight distributed computing framework for scientific workloads across heterogeneous machines, without requiring dedicated cluster infrastructure or complex deployment. It provides a central coordinator ("Hive") and connected worker processes ("Bees") that execute submitted tasks and return results to the coordinator. BeeMesh is designed for scenarios where researchers want to distribute independent or lightly coupled workloads across available devices such as laptops, desktops, and remote machines.

The current BeeMesh implementation supports three execution styles. First, it can distribute independent Python loop iterations through a lightweight launch API based on `beemesh.parallel()` or the alias `beemesh.swarm()`. Second, it can distribute uploaded executables or executable wrapper scripts for single-run or parameter-sweep workflows. Third, it includes an experimental structured-grid path for tiled two-dimensional advection with ghost-cell exchange, intended as a proof of concept for future distributed PDE workflows.

Statement of need

Many scientific users have access to multiple underutilized machines but do not want to operate or depend on full-featured cluster software. Existing distributed systems such as Ray and Dask are powerful, but they typically assume a managed cluster environment or require more explicit distributed programming patterns. Large-scale volunteer systems such as BOINC target a different operating regime and involve higher operational overhead than is necessary for small collaborative multi-machine experiments.

BeeMesh addresses a narrower need: turning a local scientific workload into a distributed run across a small, heterogeneous pool of machines with minimal setup. The framework is intended for parameter sweeps, repeated case execution, teaching demonstrations, and early-stage distributed numerical experiments. The goal is not to present BeeMesh as a production-ready distributed CFD runtime, but rather as a compact research software platform for exploring volunteer execution and lightweight distributed scientific workflows.

BeeMesh occupies a middle ground between cluster-oriented frameworks and large-scale volunteer systems by enabling small-scale, ad hoc distributed execution with minimal configuration. It complements existing tools such as Ray and Dask by focusing on lightweight deployment and volunteer-style execution, while remaining significantly simpler than large-scale volunteer computing infrastructures such as BOINC (Anderson, 2004; Moritz et al., 2018; Rocklin, 2015).

The Journal of Open Source Software

Architecture and execution model

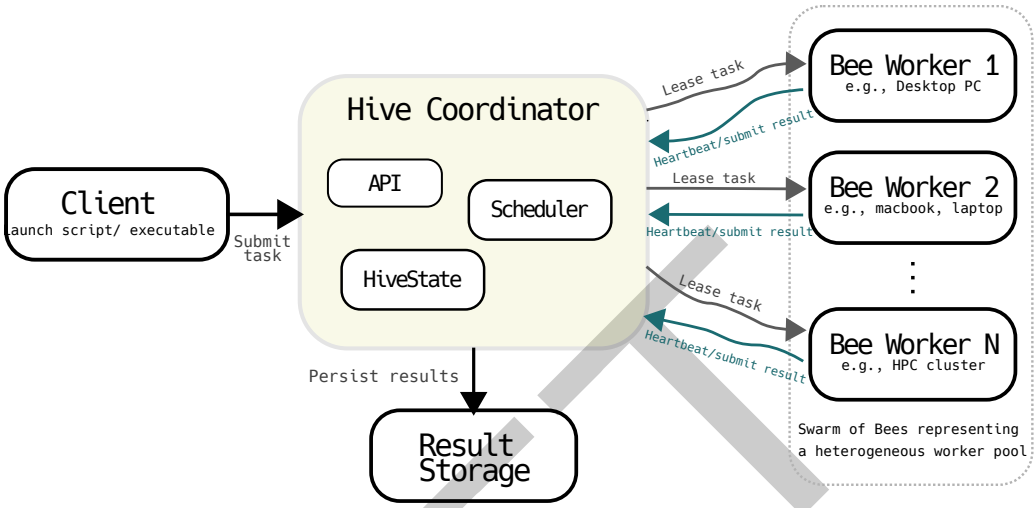

Figure 1: BeeMesh architecture: a central Hive coordinator distributes tasks to Bee workers, which execute tasks and return results.

BeeMesh follows a coordinator-worker architecture. The Hive exposes a FastAPI service for worker registration, heartbeats, task requests, result submission, and job submission. The Hive maintains in-memory runtime state, including registered workers, queued tasks, leased tasks, completed results, and worker load counters. Scheduling is handled by a dedicated module that performs hard eligibility filtering and heuristic task selection.

Bee workers register with the Hive together with basic capability metadata, including CPU cores, RAM, architecture, and optional GPU-related fields. The worker then enters a loop that requests leased tasks, executes them locally, sends heartbeats, and submits structured success or failure results back to the coordinator.

The scheduler combines several explicit heuristics:

- hard filtering for task requirements such as minimum CPU cores, minimum RAM, GPU requirements, architecture, and optional target-worker routing;
- worker-strength scoring based on resources and current task load;
- scarcity-aware prioritization so tasks that only a few workers can run are less likely to be blocked behind generic work; and
- bounded age-based task priority to reduce starvation of older queued tasks.

On top of the coordinator-worker runtime, BeeMesh includes two main user-facing launch paths. The Python launch path parses a single `beemesh.parallel()` or `beemesh.swarm()` block, evaluates the iterable locally, batches cases, and submits remote execution tasks. The executable launch path uploads a local binary or wrapper script and dispatches it as either a single-run workload or a parameter sweep. In addition, the current prototype includes an experimental grid execution path for tiled structured fields, demonstrated through a 2D advection example with ghost-cell exchange.

Capabilities and included examples

The repository includes several examples that demonstrate the current design space of BeeMesh:

- a simple distributed Python case sweep;
- a Monte Carlo parameter sweep;

DRAFT

- a neural-network hyperparameter search;
- a tiled Mandelbrot renderer with optional live preview;
- an experimental two-dimensional advection example with tiled domain decomposition and ghost-cell exchange; and
- executable launch examples for both single-run and parameter-sweep native workflows.

These examples are important to the software contribution because they show that BeeMesh is not limited to a single workload type. Instead, the framework can orchestrate independent Python tasks, native executable tasks, and an experimental mesh-coupled PDE workflow within a unified coordinator-worker runtime.

Quality control

BeeMesh includes unit tests for the core runtime components. The current test suite covers scheduler eligibility and prioritization behavior, Hive state invariants, server-side request validation and task ownership checks, launch preflight logic, and worker-side executor behavior. Continuous integration is configured through GitHub Actions to run the test suite across multiple Python versions.

The test suite is intentionally focused on runtime invariants rather than full multi-machine end-to-end orchestration. Remote workflows, including VPN-based execution, are currently validated manually in addition to automated unit-level coverage. The repository also includes example scripts that serve as functional tests for the principal execution paths.

Limitations and future work

BeeMesh remains early-stage research software. The Hive currently stores its runtime state in memory, so coordinator restarts do not preserve active jobs. GPU detection on workers is still minimal, which limits the current usefulness of GPU-aware scheduling. The structured-grid PDE workflow is experimental and is presently implemented only for the included advection example. That path uses Hive-mediated synchronization between timesteps rather than persistent subdomain-resident workers or direct worker-to-worker halo exchange.

Future work includes persistent job and state storage, richer hardware detection, more general PDE kernels, longer-lived distributed subdomain execution, and expanded fault tolerance for coupled numerical workloads.

Acknowledgements

The authors acknowledge the open-source scientific Python ecosystem, including FastAPI, NumPy, and pytest, which BeeMesh builds upon.

AI usage disclosure

OpenAI Codex/GPT-based tools were used for limited assistance with minor bug fixes, documentation and README editing, and drafting some portions of the manuscript text. Human authors reviewed, edited, and validated all AI-assisted outputs, made the substantive software and manuscript decisions, and assume full responsibility for the correctness, originality, licensing, and policy compliance of the submission.

Anderson, D. P. (2004). BOINC: A system for public-resource computing and storage. *Proceedings of the 5th IEEE/ACM International Workshop on Grid Computing*, 4–10. https://boinc.berkeley.edu/grid_paper_04.pdf

DRAFT

Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., & Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 561–577. https://www.usenix.org/conference/osdi18/presentation/moritz

Rocklin, M. (2015). Dask: Parallel computation with blocked algorithms and task scheduling. In K. Huff & J. Bergstra (Eds.), *Proceedings of the 14th python in science conference* (pp. 130–136). https://doi.org/10.25080/Majora-7b98e3ed-013

DRAFT